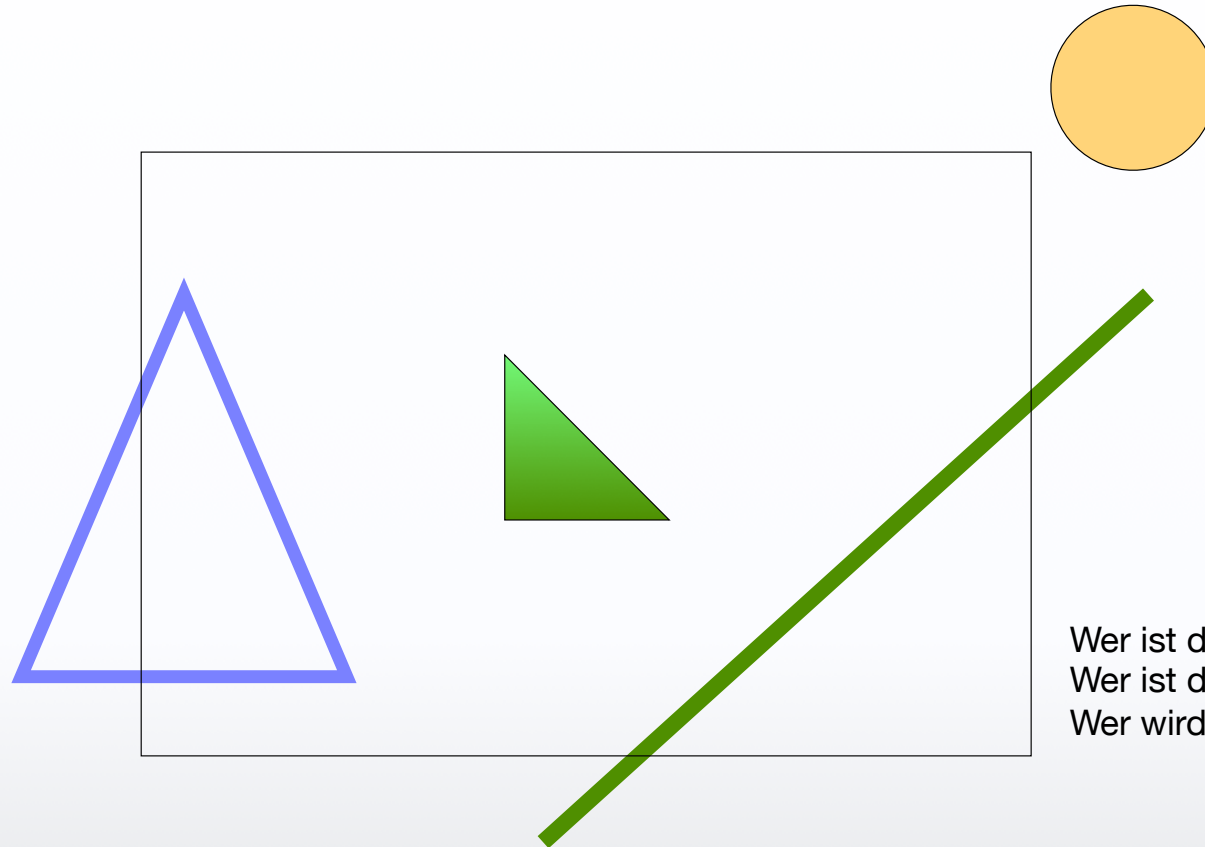


CLIPPING

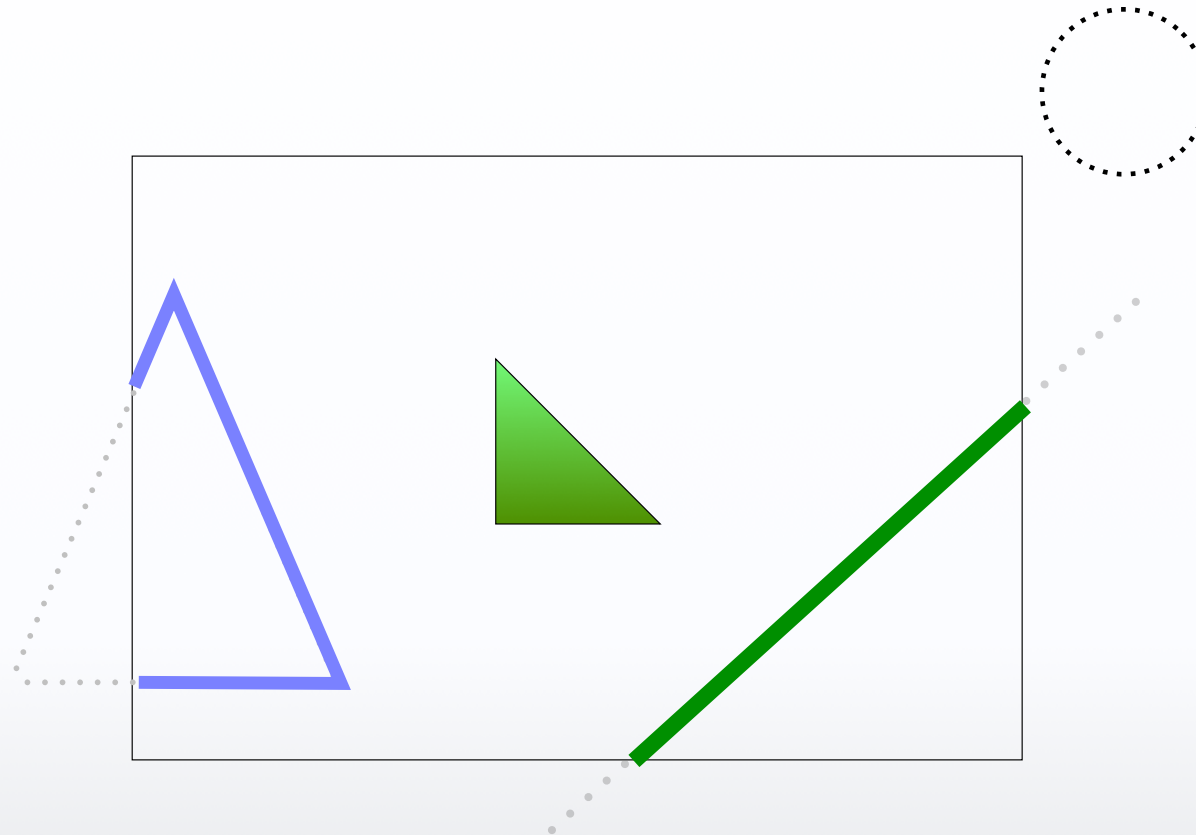


Linien und Flächen clippen

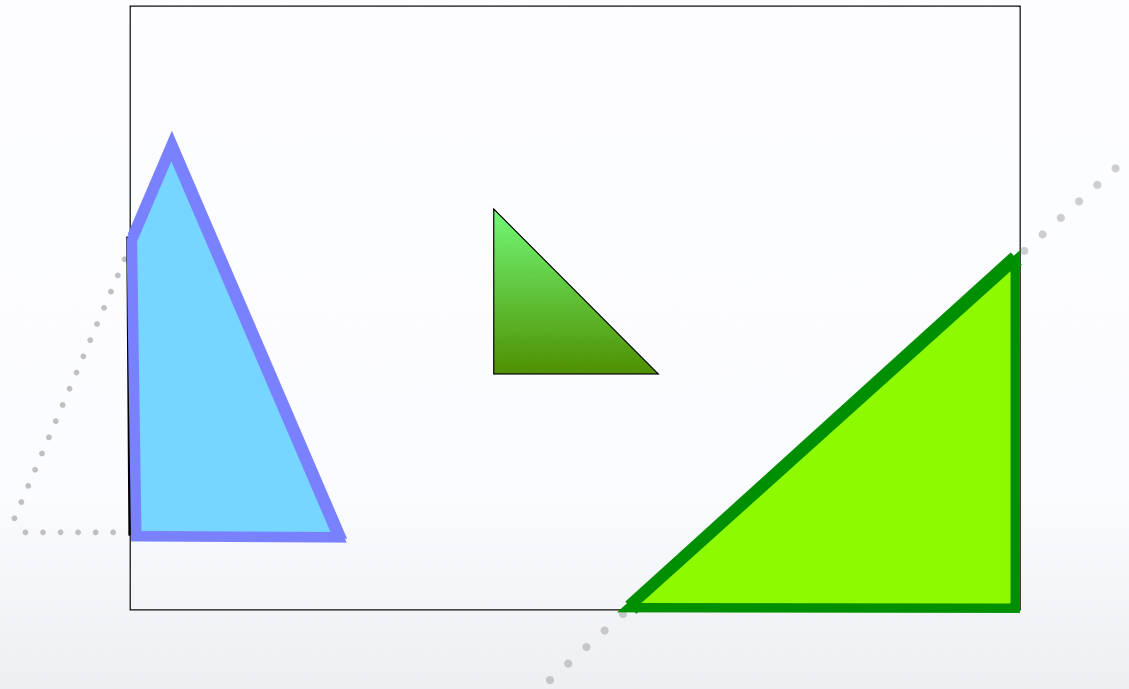


Wer ist draussen ?
Wer ist drinnen ?
Wer wird geclippt?

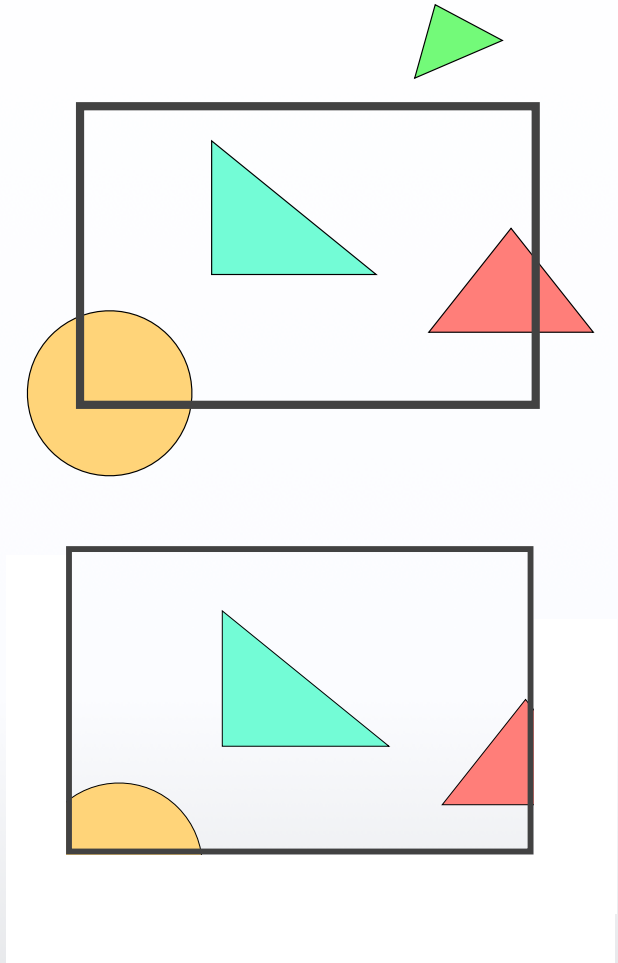
Linien und Flächen clippen



Linien und Flächen clippen



- Allgemein:
 - Objekte, Teilobjekte ausserhalb des Sichtfensters erkennen und verwerfen
- Methoden unterscheiden sich durch Art der Objekte
 - Punkte, Linien, Polygone, Kurven, Text
- Generell:
 - Objekt liegt vollständig drin
 - akzeptieren
 - Objekt liegt vollständig draussen
 - verwerfen
 - Objekt liegt zum Teil drin
 - Zerteilen und wieder testen
- Effizienz
 - Schnittpunkte minimieren



Clipping Algorithmus

2D Point clipping

- der einfachste Fall:

$$x_{wmin} \leq x \leq x_{wmax}$$

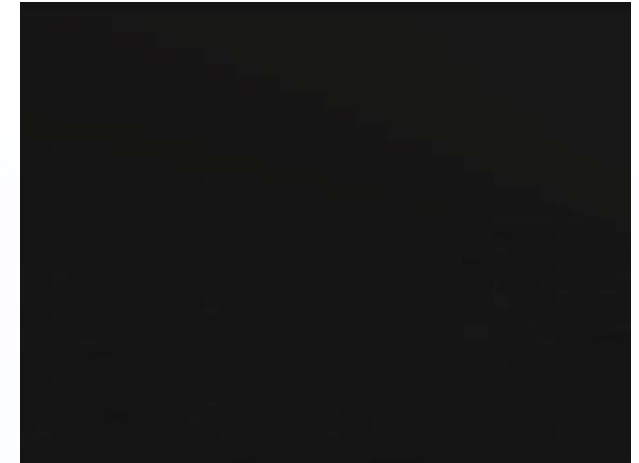
$$y_{wmin} \leq y \leq y_{wmax}$$

- einer der Ungleichungen nicht erfüllt

- Punkt verwerfen

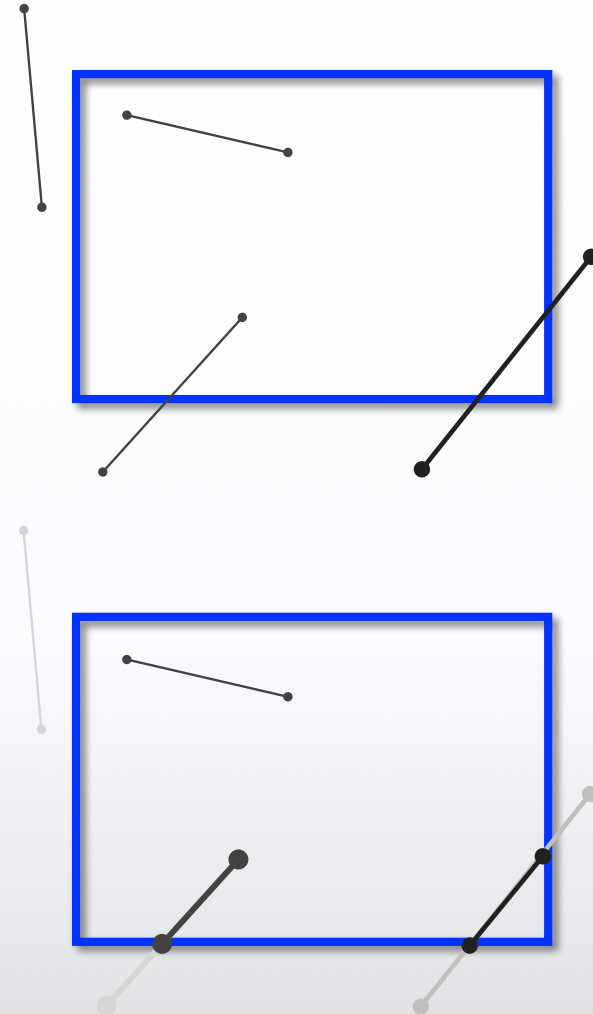
- zu verwenden bei Partikelsystemen:

- Wolken
- Rauch
- Wasser
- Feuer
- Explosionen

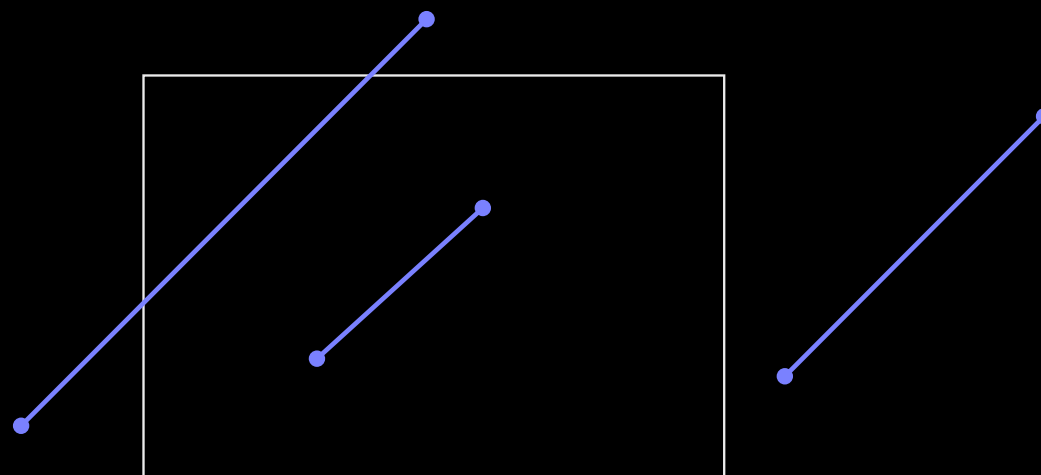


2D Line Clipping

- Analog zu „Punkten“, feststellen ob Endpunkte aussen oder innen sind
- Beide Punkte sind aussen
 - Linie verwerfen
- Beide Punkte sind innen
 - Linie zeichnen
- Wenn beide Abfragen verneint sind:
 - Linie schneidet mindestens eine Begrenzung
 - Schnittpunkte feststellen

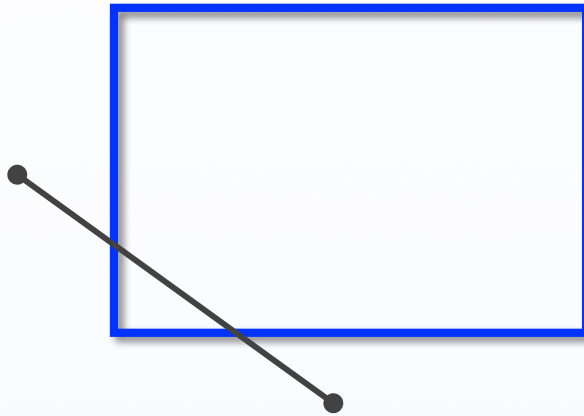


Welcher Fall ist problematisch

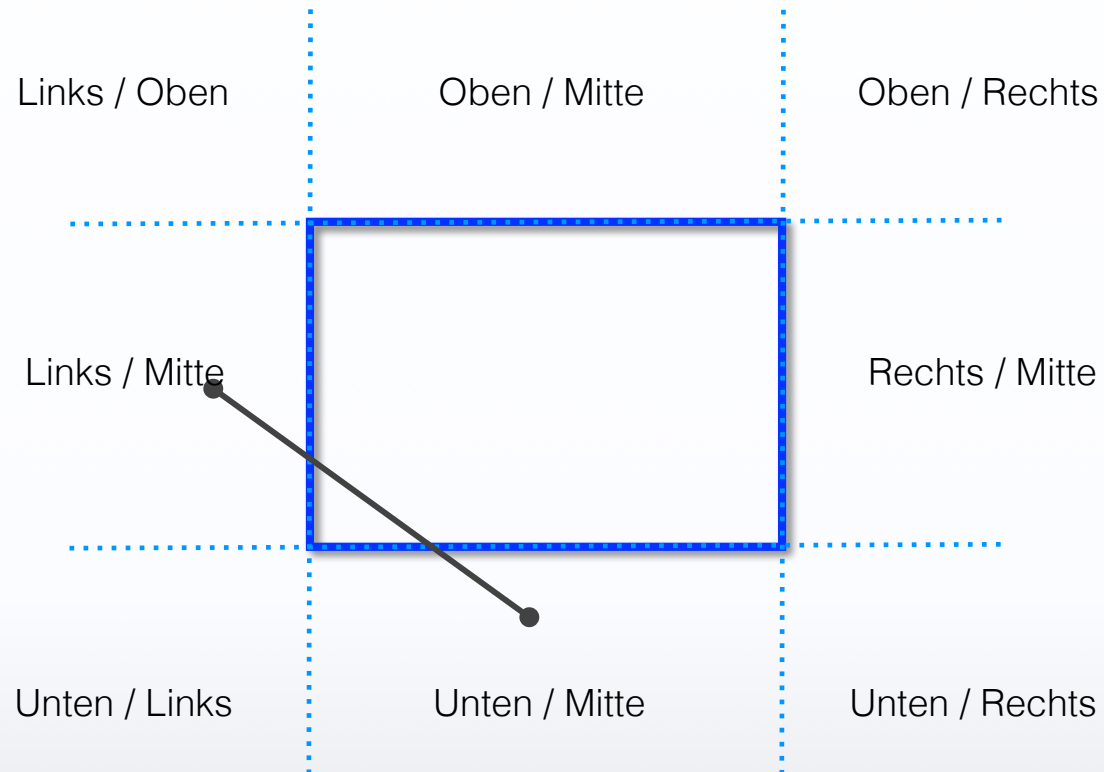


IDEE

Die Idee ?

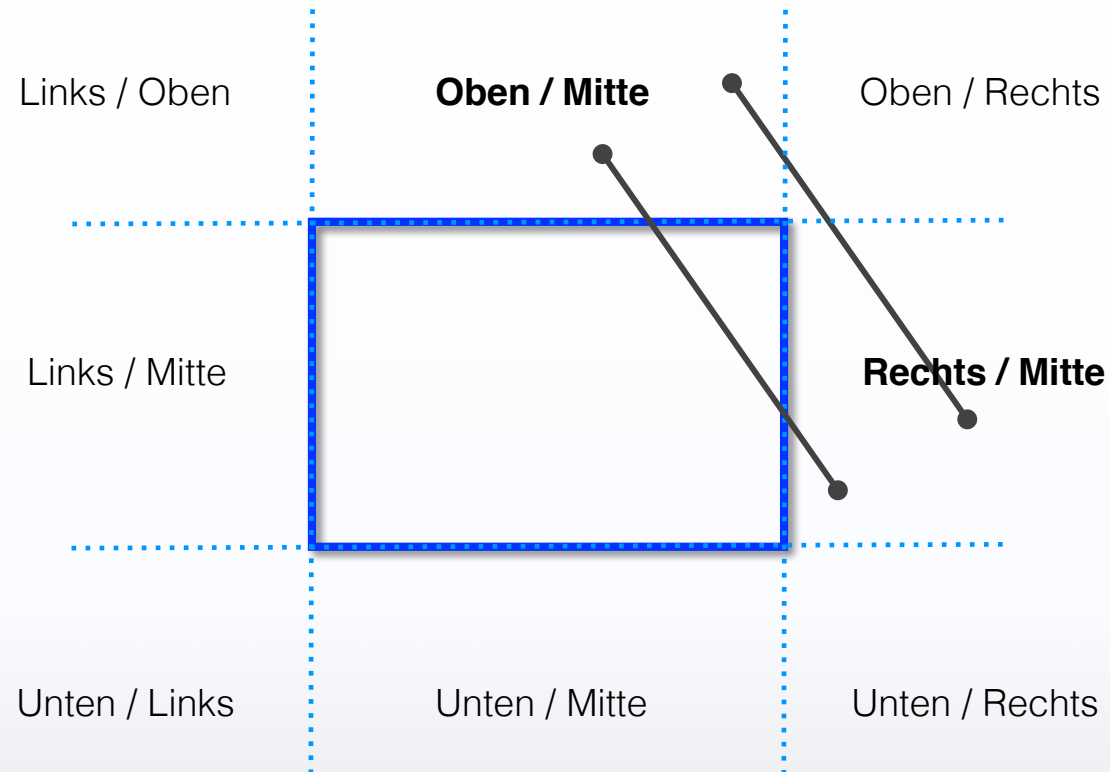


1. Regionen festlegen

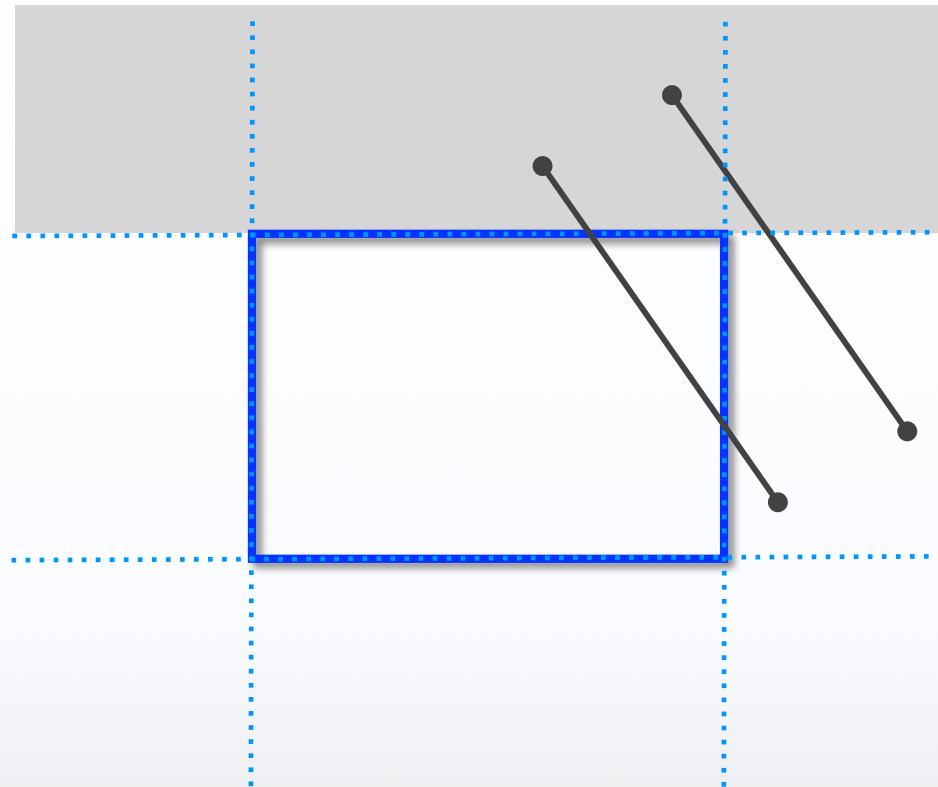


Löst das alle Probleme

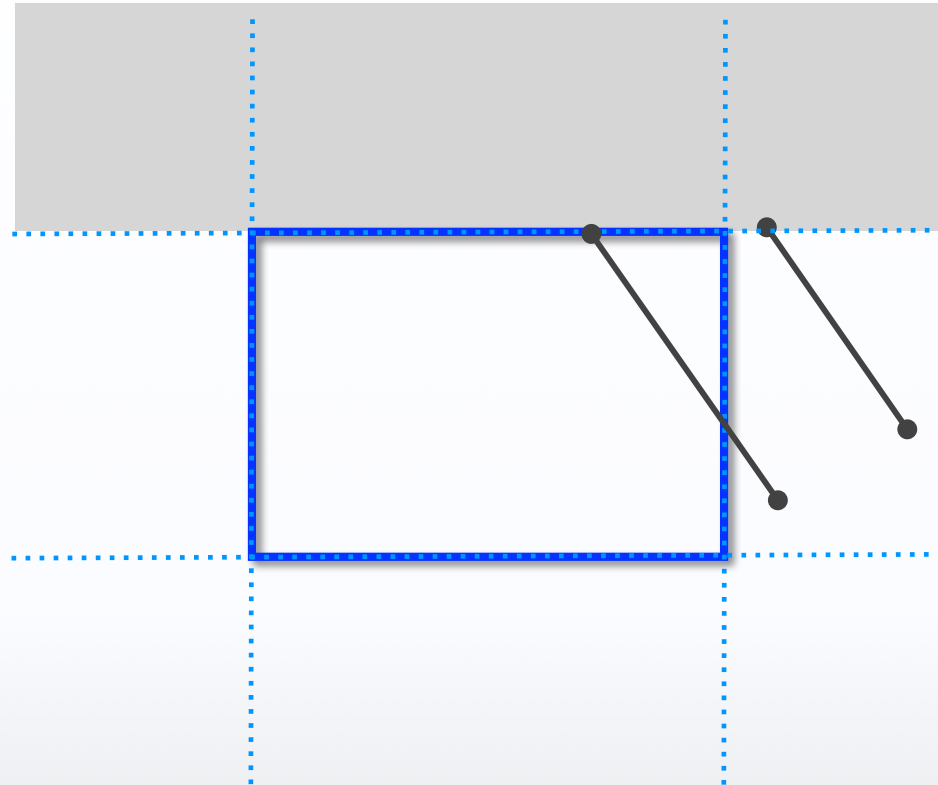
1. Regionen festlegen



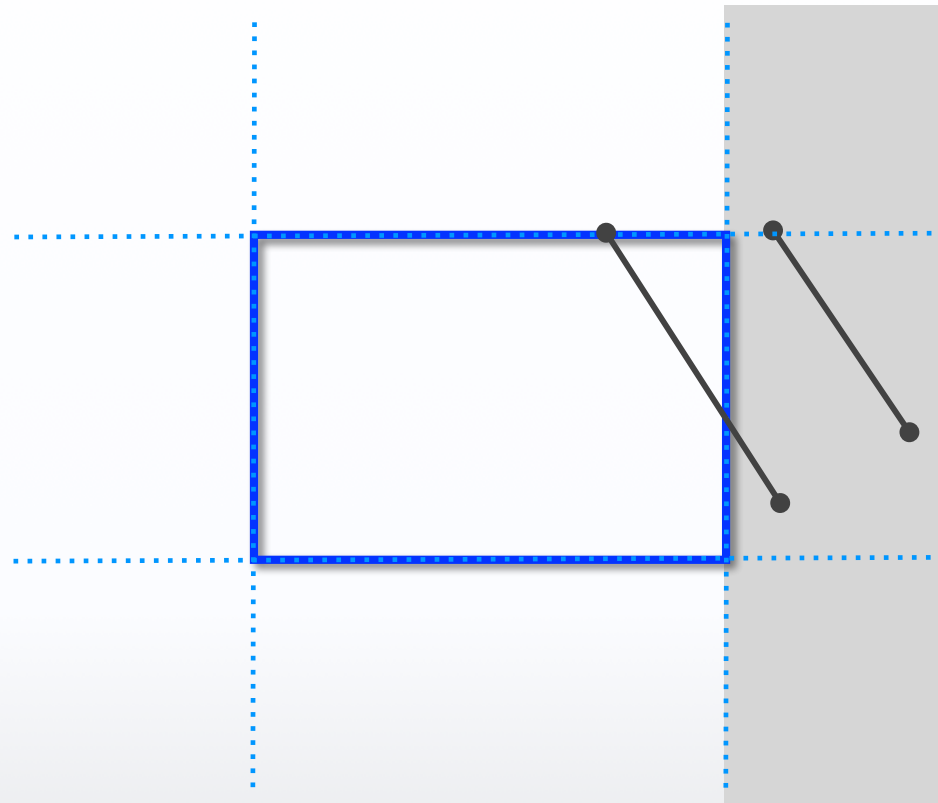
2. Sequentiell abfragen



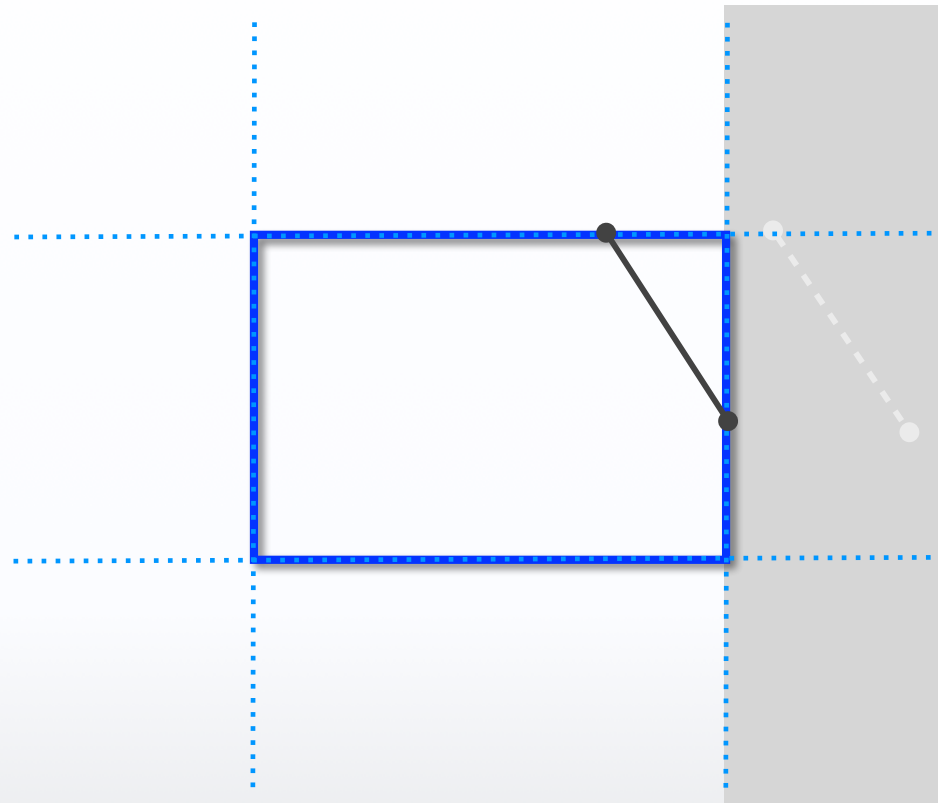
2. Sequentiell abfragen



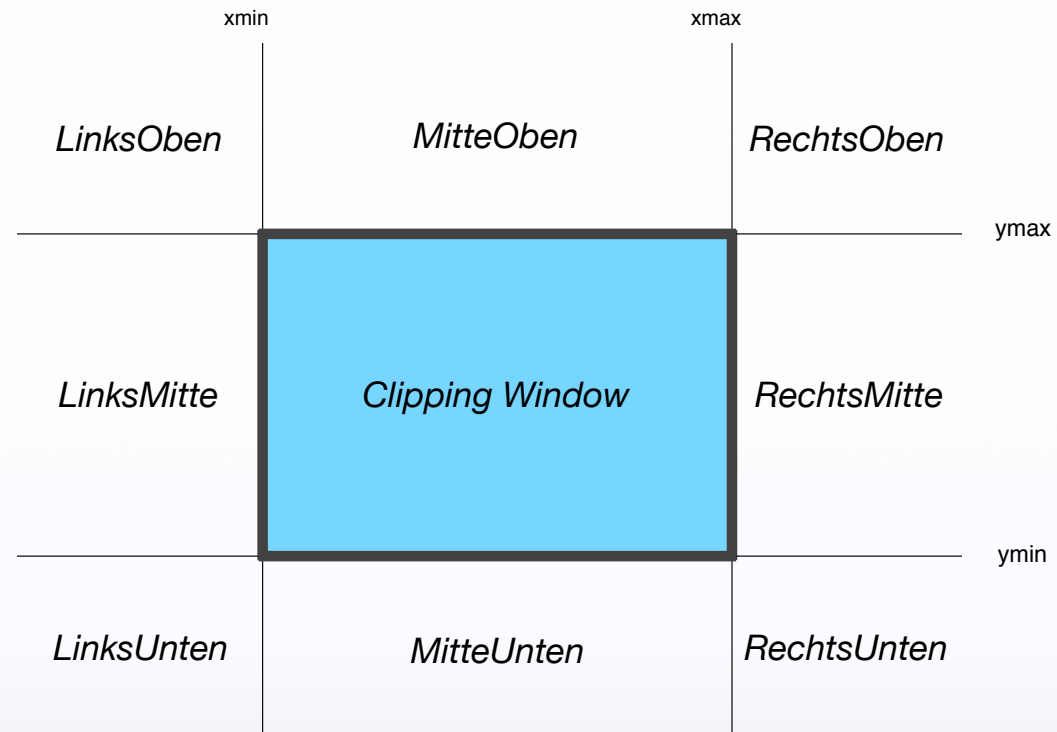
2. Sequentiell abfragen



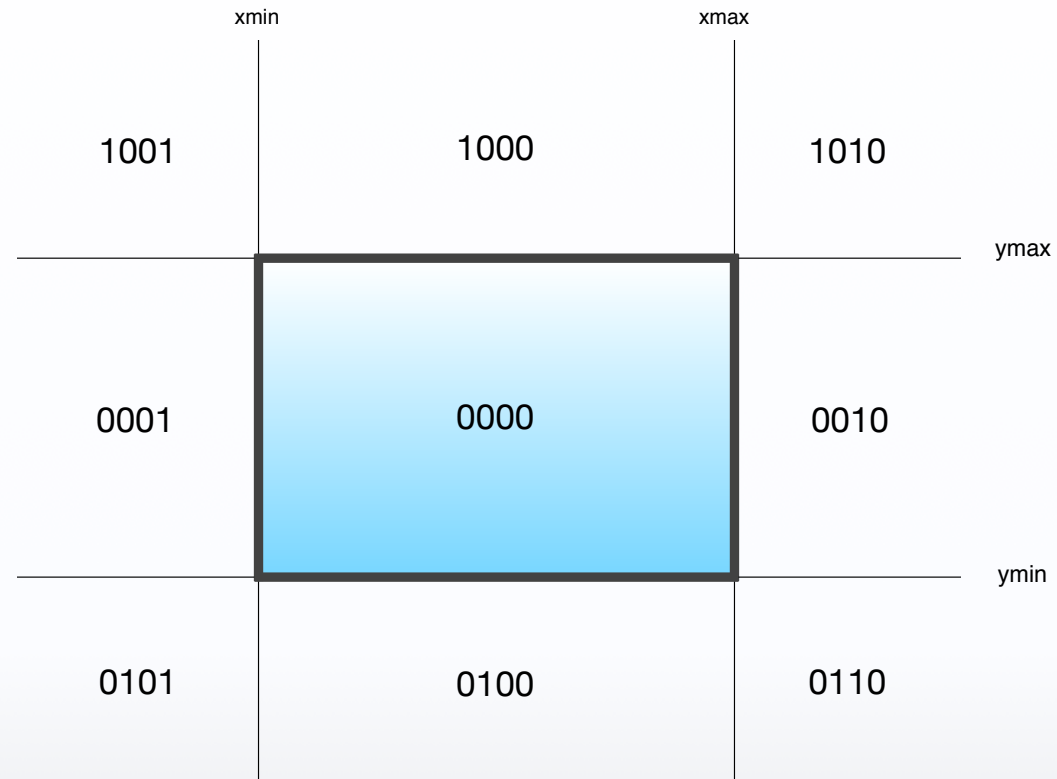
2. Sequentiell abfragen



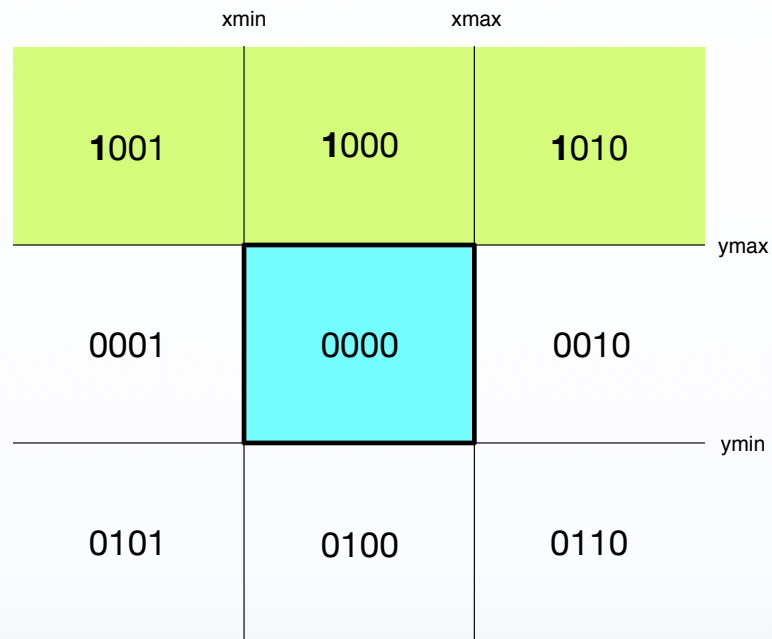
- 1. Regionen festlegen***
- 2. Abfragen in welcher Region sich ein Start- & Endpunkt befinden***
- 3. Sequentiell gegen die Regionen „clippen“***
- 4. Schnittpunkt berechnen***



Region Code

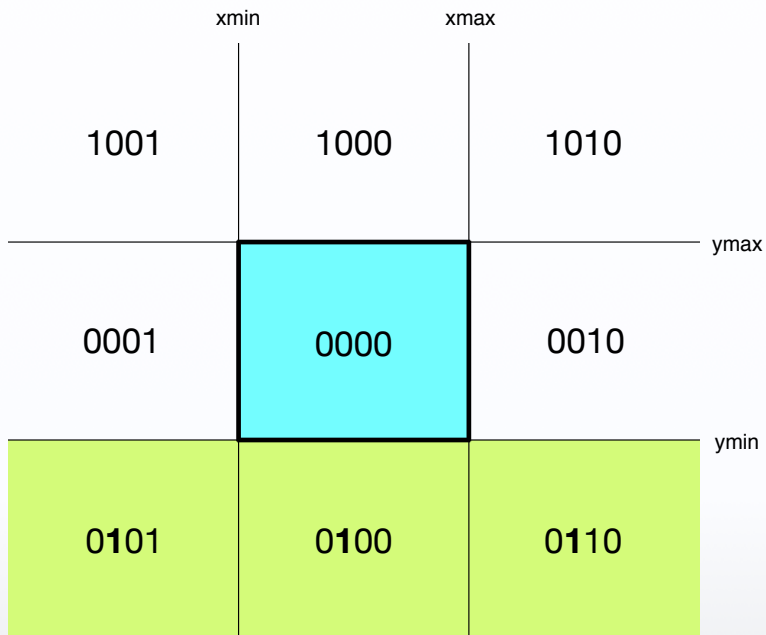


Region Code • TOP



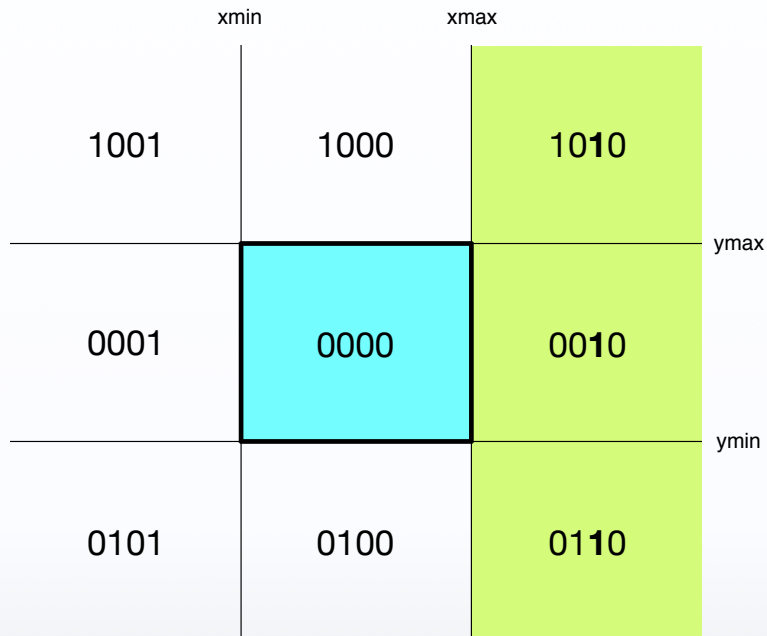
> ymax	<ymin	>xmax	<xmin
Top	Bottom	Right	Left
Punkt liegt oberhalb vom CW	Punkt liegt unterhalb vom CW	Punkt liegt rechts vom CW	Punkt liegt links vom CW
bit 4	bit 3	bit 2	bit 1

Region Code • BOTTOM



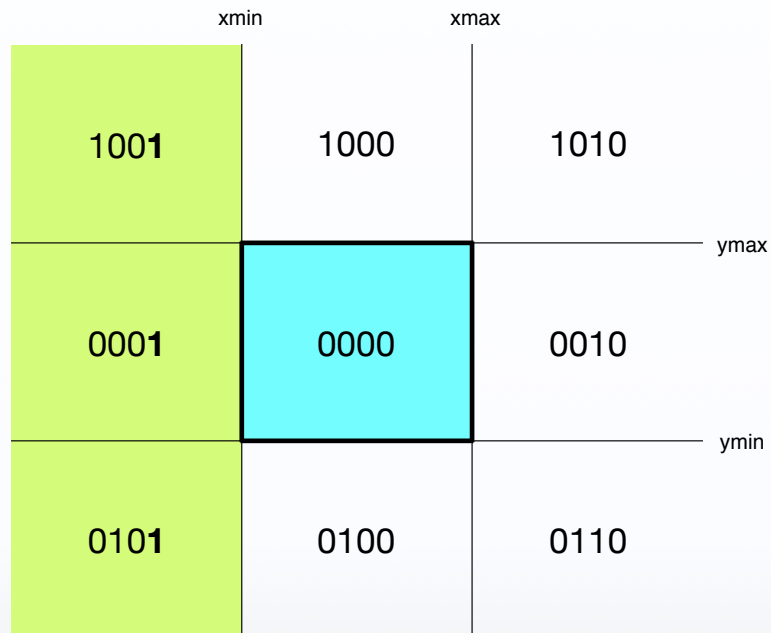
>ymax	< ymin	>xmax	<xmin
Top	Bottom	Right	Left
Punkt liegt oberhalb vom CW	Punkt liegt unterhalb vom CW	Punkt liegt rechts vom CW	Punkt liegt links vom CW
bit 4	bit 3	bit 2	bit 1

Region Code • RIGHT



>ymax	<ymin	> xmax	<xmin
Top	Bottom	Right	Left
Punkt liegt oberhalb vom CW	Punkt liegt unterhalb vom CW	Punkt liegt rechts vom CW	Punkt liegt links vom CW
bit 4	bit 3	bit 2	bit 1

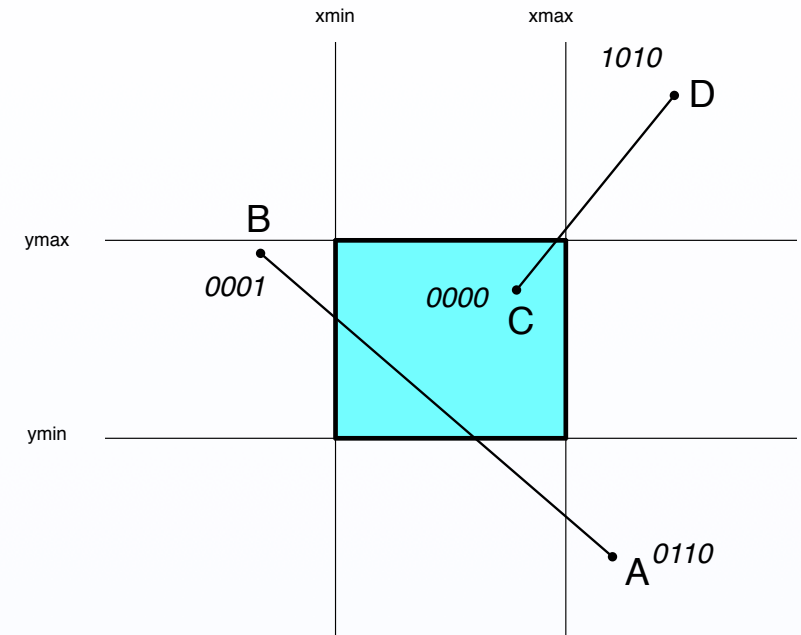
Region Code • LEFT



>ymax	<ymin	>xmax	< xmin
Top	Bottom	Right	Left
Punkt liegt oberhalb vom CW	Punkt liegt unterhalb vom CW	Punkt liegt rechts vom CW	Punkt liegt links vom CW
bit 4	bit 3	bit 2	bit 1

Region Code erstellen

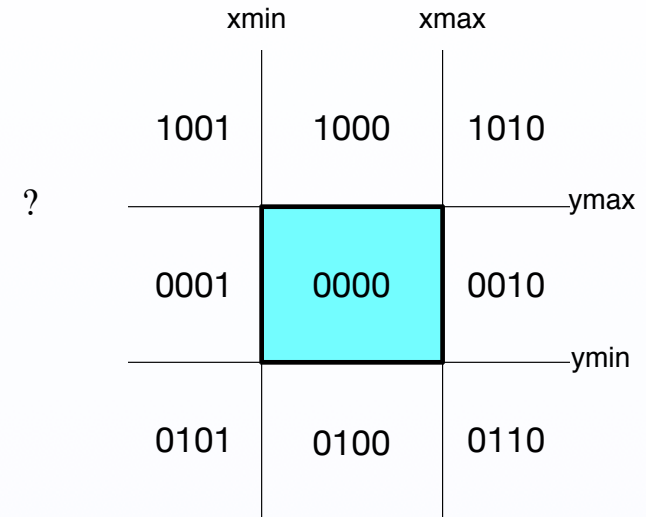
	x	y	BIT4	BIT3	BIT2	BIT1
A	32	-21	0	1	1	0
B	-21	12	0	0	0	1
C	4	9	0	0	0	0
D	20	36	1	0	1	0



Top	Bottom	Right	Left
BIT 4	BIT 3	BIT 2	BIT 1

Region Code Code

```
unsigned char regionCode (int x, int y){  
    unsigned char c =0;  
  
    if (y > ymax) c = c | 8; // 1000 Top  
    if (y < ymin) c = c | 4; // 0100 Bottom  
  
    if (x > xmax) c = c | 2; // 0010 Right  
    if (x < xmin) c = c | 1; // 0001 Left  
  
    return c;  
}
```



Top	Bottom	Right	Left
bit 4	bit 3	bit 2	bit 1



unsigned char / Zahlenbereich? 0-255

Cohen Sutherland Line Clipping

- Anfangs- und Endpunkt sind zugeordnet
- Identifizierung der Linien die klar innen sind
 - draw
- Identifizierung der Linien die klar aussen sind
 - reject
- Identifizieren der Clippingkandidaten
- Sequentielles Clipping gegen jeden Clipper
- Berechnung der Schnittpunkte
- Aber wie?
- Was können wir mit der BIT-Informationen anfangen?

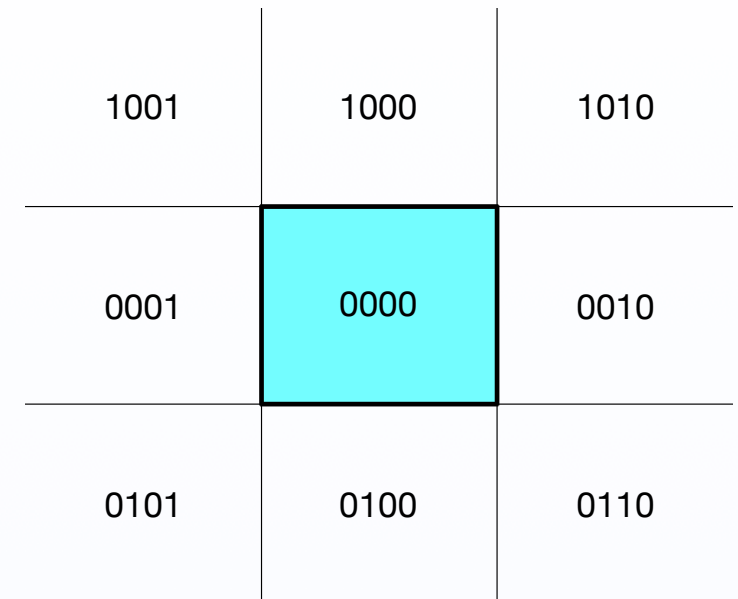
1001	1000	1010
0001	0000	0010
0101	0100	0110

Top	Bottom	Right	Left
bit 4	bit 3	bit 2	bit 1

Cohen Sutherland Line Clipping

Anfangs- und Endpunkt logisch verknüpfen

- 1 Bitweise OR Verknüpfung ergibt 0000
- 2 Bitweise UND Verknüpfung ergibt NICHT 0000
- 3 Bitweise UND Verknüpfung ergibt 0000



Top	Bottom	right	left
bit 4	bit 3	bit 2	bit 1

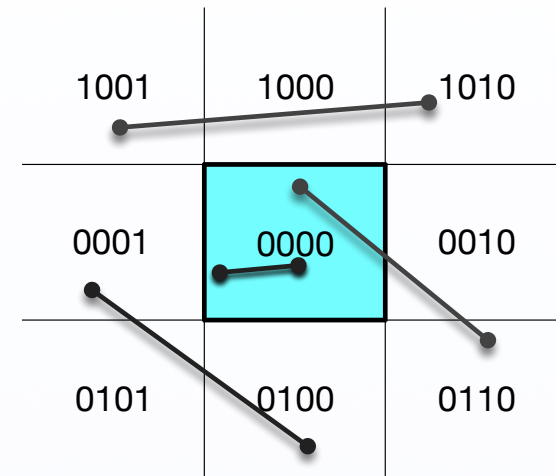
Cohen Sutherland Line Clipping

1

Bitweise OR ergibt 0000

- a) Linie vollständig draussen
- b) Linie vollständig drinnen
- c) Linie clippen

b) Linie vollständig drinnen
Zeichnen!



Top	Bottom	right	left
bit 4	bit 3	bit 2	bit 1

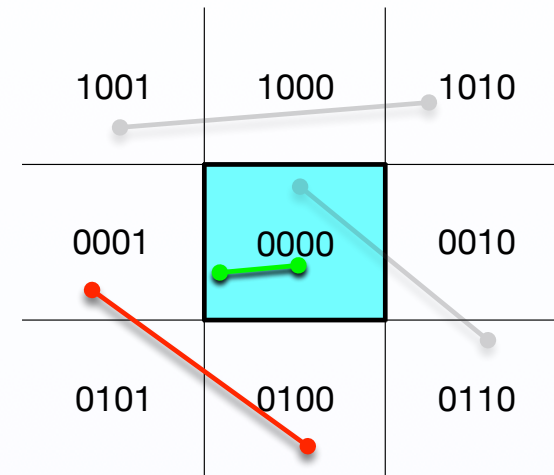
Cohen Sutherland Line Clipping

1

Bitweise OR ergibt 0000

- a) Linie vollständig draussen
- b) Linie vollständig drinnen
- c) Linie clippen

b) Linie vollständig drinnen
Zeichnen!



Top	Bottom	right	left
bit 4	bit 3	bit 2	bit 1

OR

0	0	0	0
0	0	0	0
0	0	0	0

OR

0	0	0	1
0	1	0	0
0	1	0	1

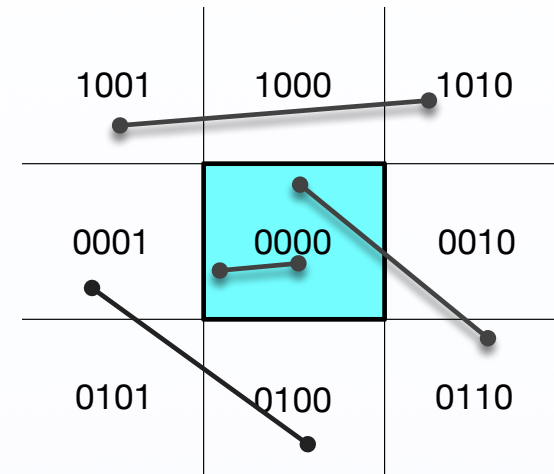
Cohen Sutherland Line Clipping

2

Bitweise AND ergibt NICHT 0000

- a) Linie vollständig draussen
- b) Linie vollständig drinnen
- c) Linie clippen

a) Linie vollständig draussen
Verwerfen!



Top	Bottom	right	left
bit 4	bit 3	bit 2	bit 1

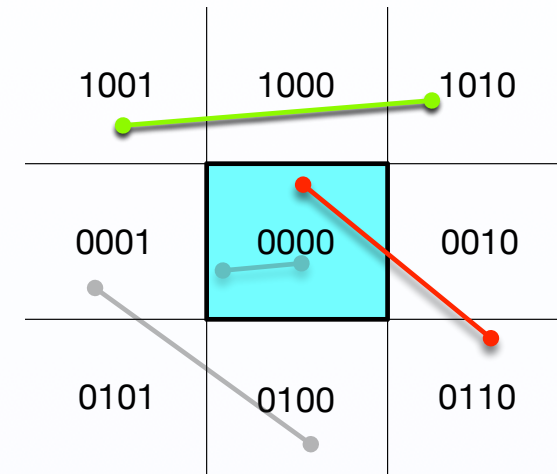
Cohen Sutherland Line Clipping

2

Bitweise AND ergibt NICHT 0000

- a) Linie vollständig draussen
- b) Linie vollständig drinnen
- c) Linie clippen

- a) Linie vollständig draussen
Verwerfen!



Top	Bottom	right	left
bit 4	bit 3	bit 2	bit 1

AND

1	0	0	1
1	0	1	0
1	0	0	0

AND

0	0	0	0
0	1	1	0
0	0	0	0

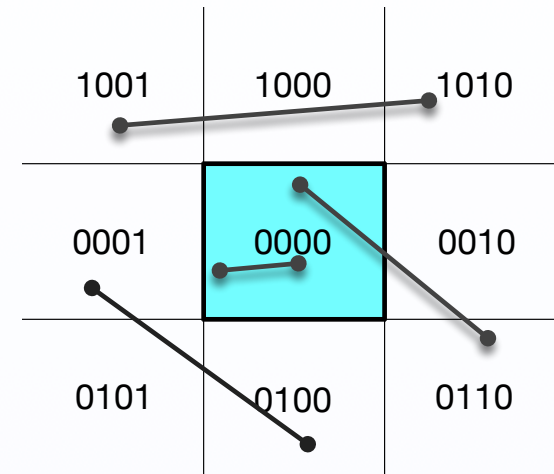
Cohen Sutherland Line Clipping

3

Bitweise UND ergibt 0000

- a) Linie vollständig draussen
- b) Linie vollständig drinnen
- c) Linie clippen

c) Linie ist teilweise innen; CLIPPEN !
Wiederholen !



Top	Bottom	right	left
bit 4	bit 3	bit 2	bit 1

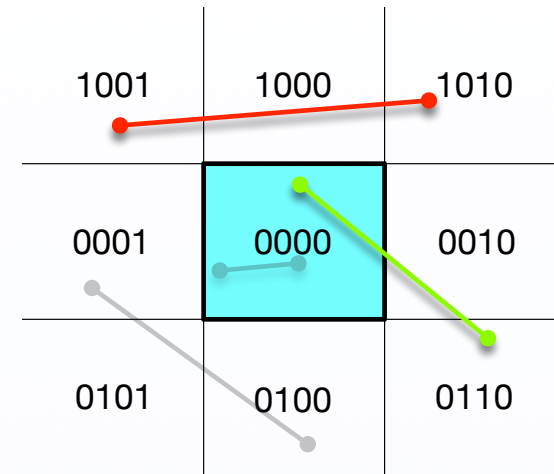
Cohen Sutherland Line Clipping

3

Bitweise UND ergibt 0000

- a) Linie vollständig draussen
- b) Linie vollständig drinnen
- c) Linie clippen

c) Linie ist teilweise innen; CLIPPEN !
Wiederholen !



Top	Bottom	right	left
bit 4	bit 3	bit 2	bit 1

AND

0	0	0	0
0	1	1	0
0	0	0	0

AND

1	0	0	1
1	0	1	0
1	0	0	0

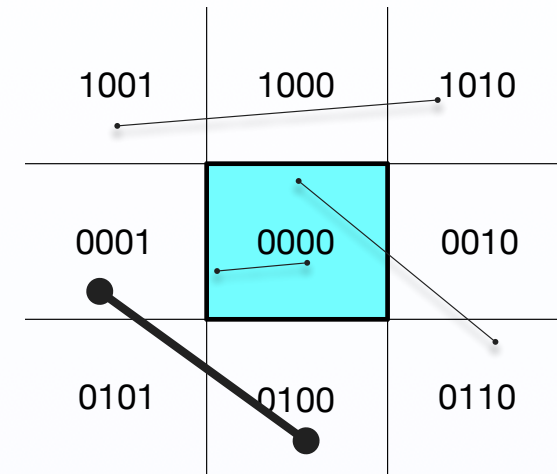
Cohen Sutherland Line Clipping

3

Bitweise UND ergibt 0000

- a) Linie vollständig draussen
- b) Linie vollständig drinnen
- c) Linie clippen

c) Linie ist teilweise innen; **CLIPPEN !**
Wiederholen !



Top	Bottom	right	left
bit 4	bit 3	bit 2	bit 1

AND

0	0	0	1
0	1	0	0
0	0	0	0

Wir stellen nur fest, welche Linien auf jeden Fall nicht geclippt werden müssen.

Cohen Sutherland Line Clipping

- Identifizieren der Linien die klar innen oder aussen sind.
- Identifizieren der Clippingkandidaten

(if) Bitweise OR Verknüpfung 0000 ergibt

Linie ist komplett Innen. Zeichnen

(if) Bitweise UND Verknüpfung nicht 0000 ergibt

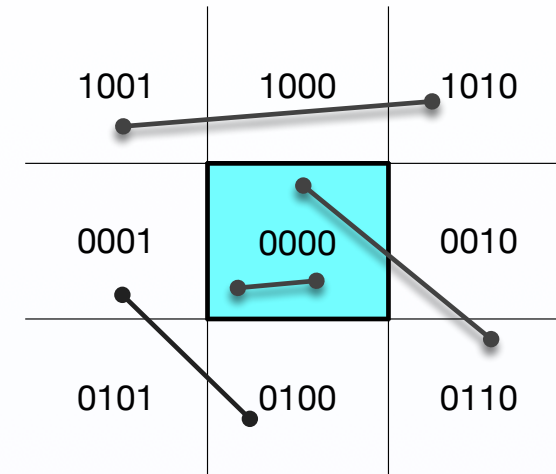
Linie ist komplett aussen. Verwerfen

(else) Bitweise UND Verknüpfung 0000 ist

Clipping Kandidat.

Sequentielles Clipping gegen jeden Clipper

Schnittpunkte berechnen.



Top	Bottom	right	left
bit 4	bit 3	bit 2	bit 1

Cohen Sutherland Line Clipping

■ Linien klassifizieren

- Linie a: [0001, 0100], Linie b: [0000, 0000]
- Linie c: [0000, 1000], Linie d: [0110, 1010]

■ Linien testen:

■ a:

- $0001 \text{ OR } 0100 = 0101$: nicht 0000, weiter
- $0001 \text{ AND } 0100 = 0000$: Clipping Kandidat

■ b:

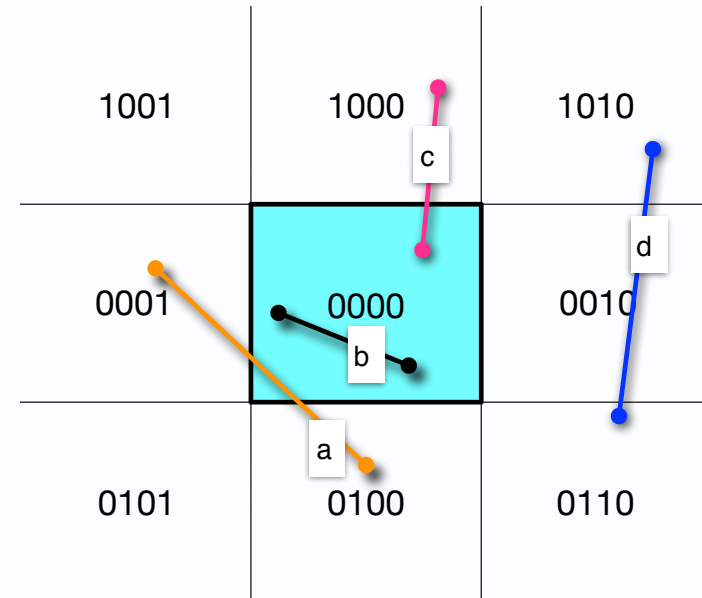
- $0000 \text{ OR } 0000 = 0000$ Linie zeichnen

■ c:

- $1000 \text{ OR } 0000 = 1000$: nicht 0000, weiter
- $1000 \text{ AND } 0000 = 0000$: Clipping Kandidat

■ d:

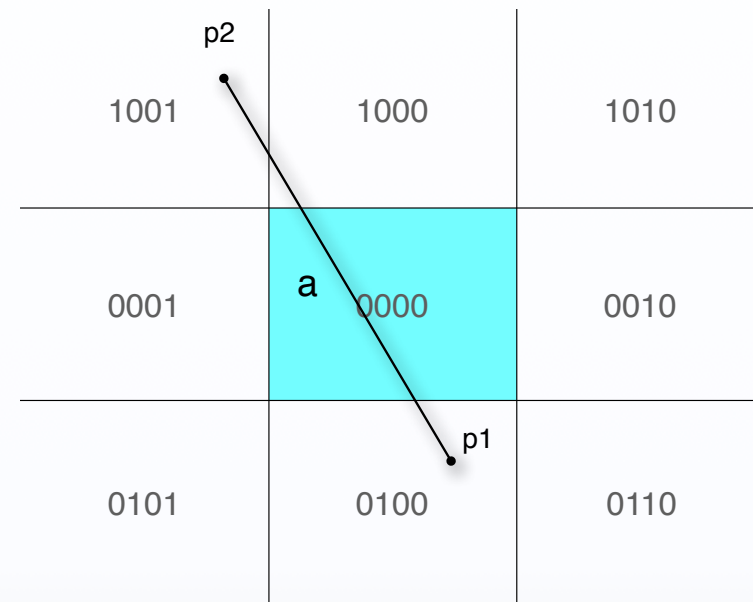
- $1010 \text{ OR } 0110 = 1110$: nicht 0000, weiter
- $1010 \text{ AND } 0110 = 0010$: nicht 0000, Verwerfen



- If (Bitwise OR = 0000)
 - draw line
- If (Bitwise AND $\neq$ 0000)
 - reject line
- (else) Bitwise AND= 0000
 - clip line

Clipping Kandidaten

- Die Punkte die nicht im CW sind (Nicht 0000)
- Gegen jede Begrenzung sequentiell abfragen:
- Left/Right/Bottom/Top:
- Wenn Intersectionpoint vorhanden
 - Clippen
- Neues Segment testen
- bis Region Code Test [0000],[0000]
- Linie zeichnen



Cohen Sutherland Line Clipping

- Clipping Kandidaten:
- Gegen jede Begrenzung sequentiell abfragen:

■ Left:

■ Linie a:

- Linie $[p1, p2]$

- Left: Intersect: $p2'$, $[p2', p2]$ clippen

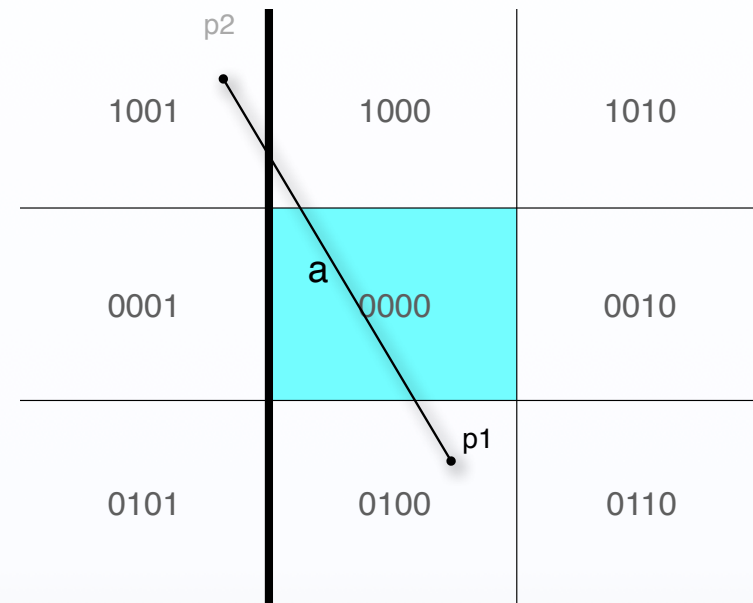
- Right[]

- Bottom: Intersect $p1'$, $[p1', p1]$ clippen

- Top: Intersect $p2''$, $[p2'', p2']$ clippen

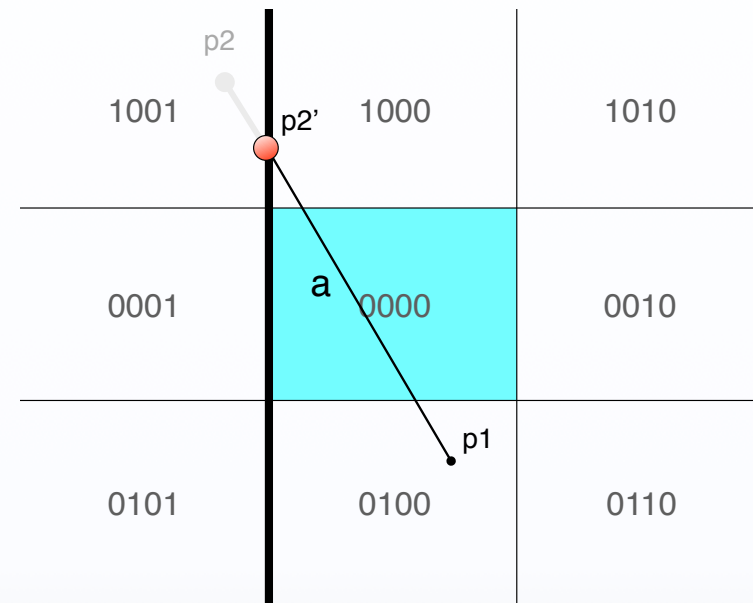
- Linie $[p2'', p2']$

- Region Code Test $[0000], [0000]$: Linie zeichnen



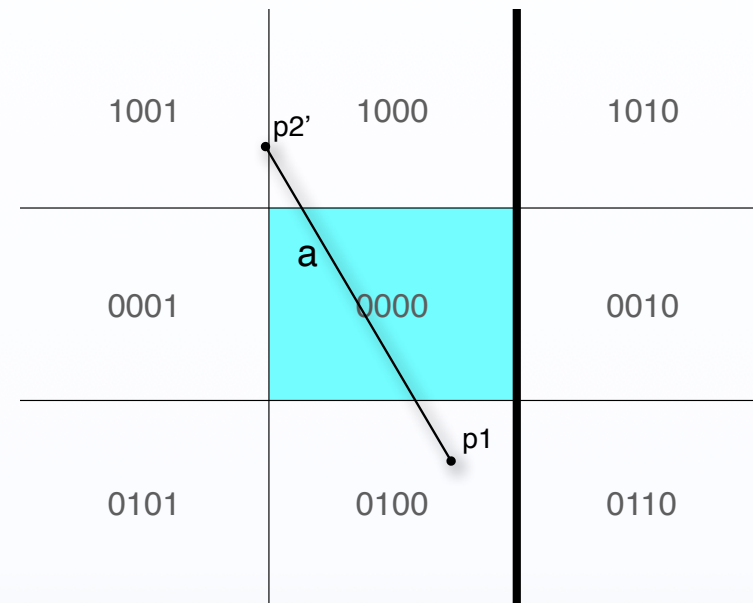
Cohen Sutherland Line Clipping

- Clipping Kandidaten:
- Gegen jede Begrenzung sequentiell abfragen:
- Left:
- Linie a:
 - Linie $[p1, p2]$
 - Left: Intersect: $p2'$, $[p2', p2]$ clippen
 - Right[]
 - Bottom: Intersect $p1'$, $[p1', p1]$ clippen
 - Top: Intersect $p2''$, $[p2'', p2']$ clippen
 - Linie $[p2'', p2']$
 - Region Code Test $[0000], [0000]$: Linie zeichnen



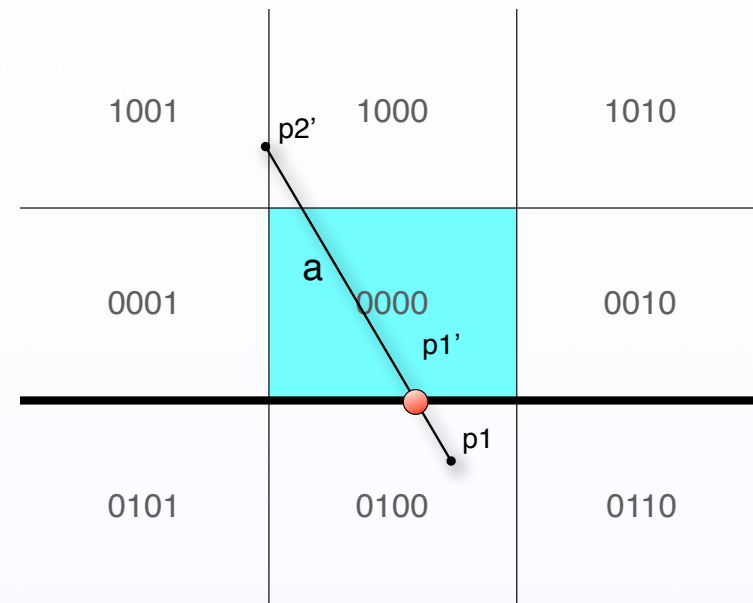
Cohen Sutherland Line Clipping

- Clipping Kandidaten:
- Gegen jede Begrenzung sequentiell abfragen:
- Right:
- Linie a:
 - Linie [p1,p2]
 - Left: Intersect: p2', [p2',p2] clippen
 - Right[]
 - Bottom: Intersect p1', [p1',p1] clippen
 - Top: Intersect p2'', [p2'',p2'] clippen
 - Linie [p2'',p2']
 - Region Code Test [0000],[0000]: Linie zeichnen



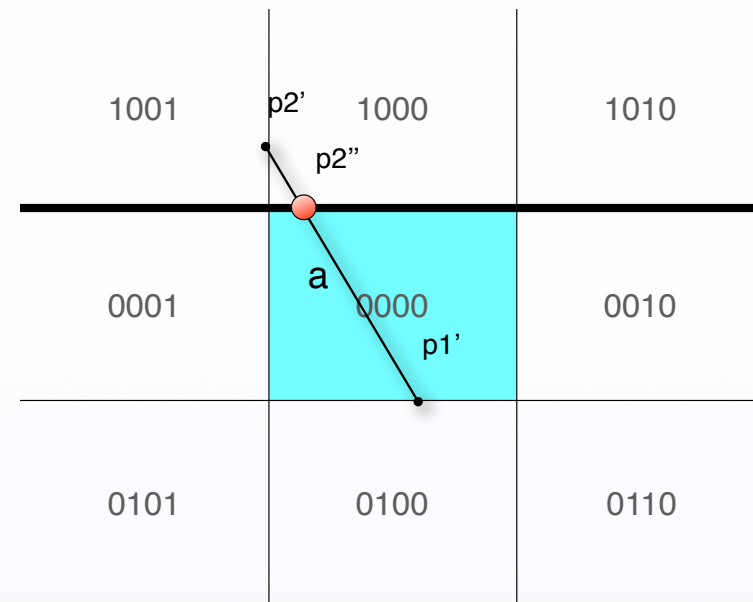
Cohen Sutherland Line Clipping

- Clipping Kandidaten:
- Gegen jede Begrenzung sequentiell abfragen:
- Left/Right/Bottom/Top:
- Linie a:
 - Linie $[p1, p2]$
 - Left: Intersect: $p2'$, $[p2', p2]$ clippen
 - Right[]
 - Bottom: Intersect $p1'$, $[p1', p1]$ clippen
 - Top: Intersect $p2''$, $[p2'', p2']$ clippen
 - Linie $[p2'', p2']$
 - Region Code Test $[0000], [0000]$: Linie zeichnen



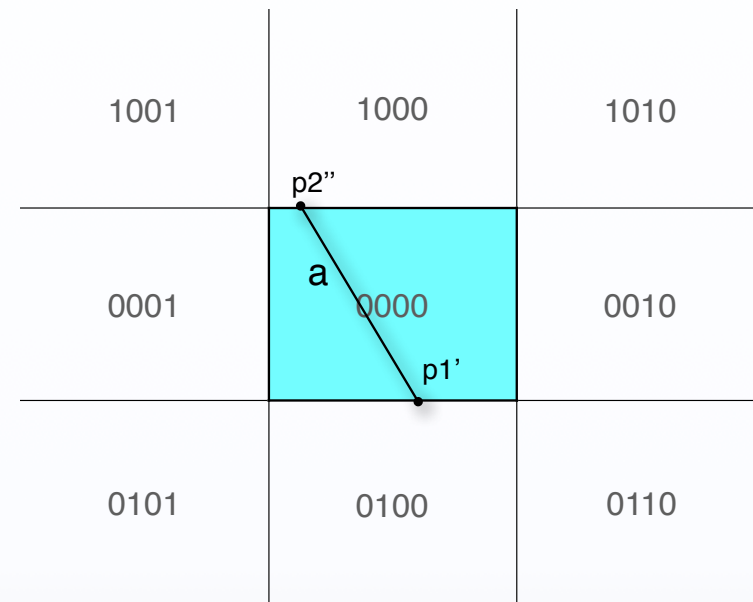
Cohen Sutherland Line Clipping

- Clipping Kandidaten:
- Gegen jede Begrenzung sequentiell abfragen:
- Left/Right/Bottom/Top:
- Linie a:
 - Linie $[p1, p2]$
 - Left: Intersect: $p2'$, $[p2', p2]$ clippen
 - Right[]
 - Bottom: Intersect $p1'$, $[p1', p1]$ clippen
 - Top: Intersect $p2''$, $[p2'', p2']$ clippen
 - Linie $[p2'', p2']$
 - Region Code Test $[0000], [0000]$: Linie zeichnen



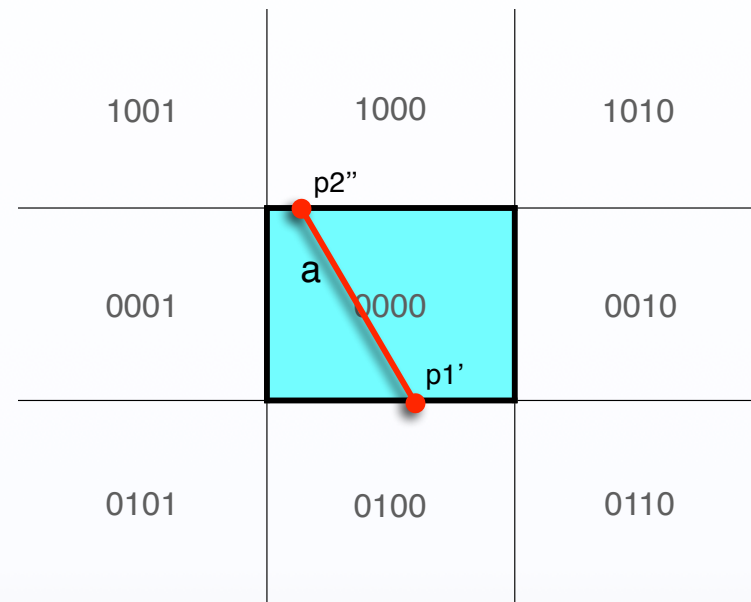
Cohen Sutherland Line Clipping

- Clipping Kandidaten:
- Gegen jede Begrenzung sequentiell abfragen:
- Left/Right/Bottom/Top:
- Linie a:
 - Linie $[p1, p2]$
 - Left: Intersect: $p2'$, $[p2', p2]$ clippen
 - Right[]
 - Bottom: Intersect $p1'$, $[p1', p1]$ clippen
 - Top: Intersect $p2''$, $[p2'', p2']$ clippen
 - Linie $[p2'', p1']$
 - Region Code Test $[0000], [0000]$: Linie zeichnen



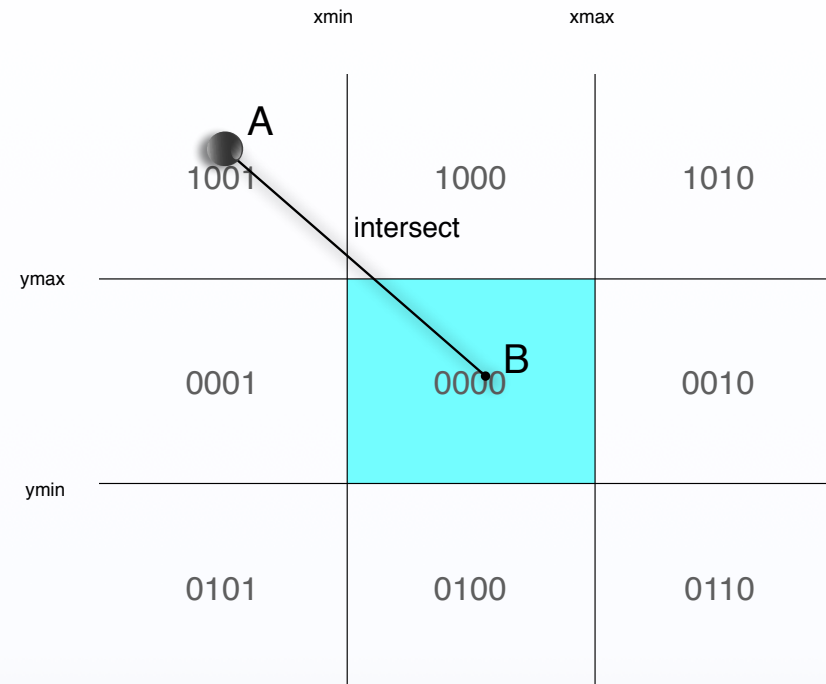
Cohen Sutherland Line Clipping

- Clipping Kandidaten:
- Gegen jede Begrenzung sequentiell abfragen:
- Left/Right/Bottom/Top:
- Linie a:
 - Linie $[p1, p2]$
 - Left: Intersect: $p2'$, $[p2', p2]$ clippen
 - Right[]
 - Bottom: Intersect $p1'$, $[p1', p1]$ clippen
 - Top: Intersect $p2''$, $[p2'', p2']$ clippen
 - Linie $[p2'', p1']$
 - Region Code Test $[0000], [0000]$: Linie zeichnen



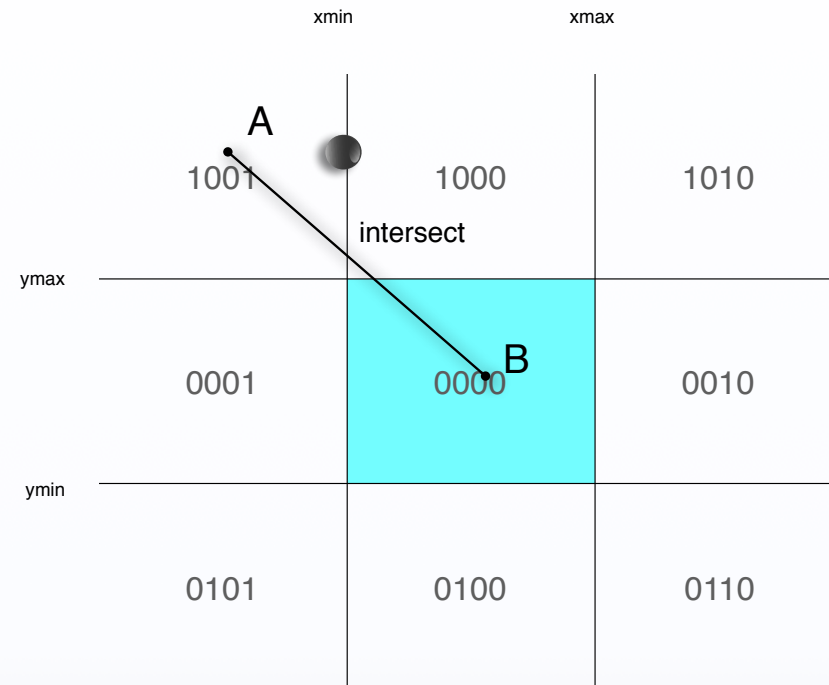
Cohen Sutherland Line Clipping

- Wie werden die Koordinaten des Schnittpunktes berechnet?
- Den Punkt auswählen dessen RegionCode ungleich 0000 ist (aussen): PUNKT A



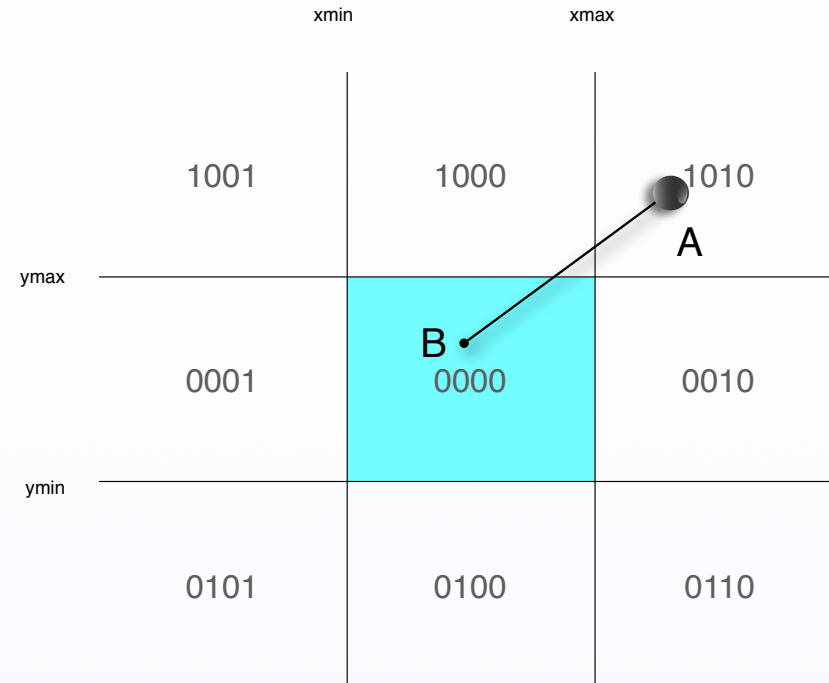
Cohen Sutherland Line Clipping

- Wie werden die Koordinaten des Schnittpunktes berechnet?
- Den Punkt auswählen dessen RegionCode ungleich 0000 ist (aussen): PUNKT A
- Bit1 gesetzt (Left):
 - Setze x-Wert= xmin



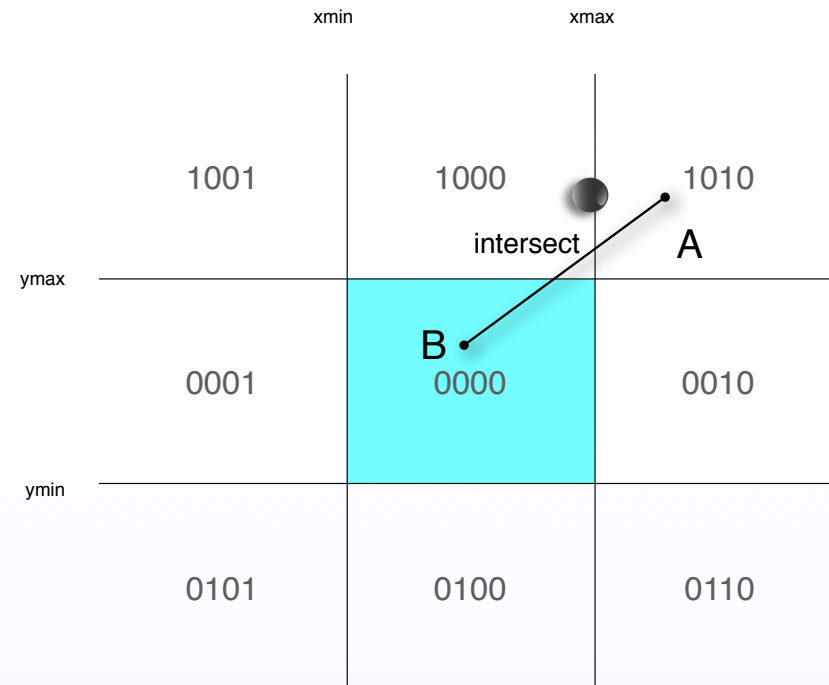
Cohen Sutherland Line Clipping

- Wie werden die Koordinaten des Schnittpunktes berechnet?
- Den Punkt auswählen dessen RegionCode ungleich 0000 ist (aussen): PUNKT A
- Bit1 gesetzt (Left):
 - Setze x-Wert= xmin
- Bit2 gesetzt (Right):



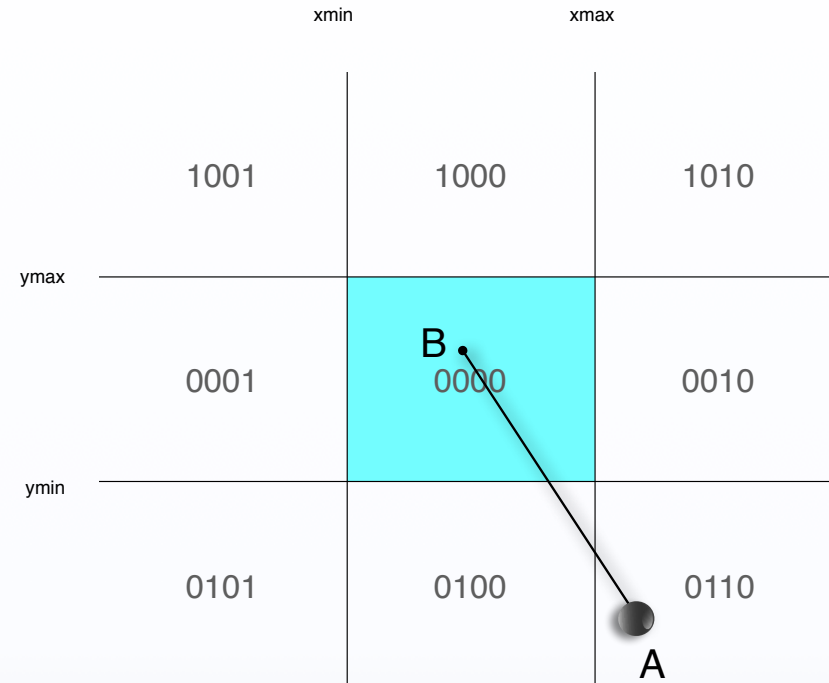
Cohen Sutherland Line Clipping

- Wie werden die Koordinaten des Schnittpunktes berechnet?
- Den Punkt auswählen dessen RegionCode ungleich 0000 ist (aussen): PUNKT A
- Bit1 gesetzt (Left):
 - Setze x-Wert= xmin
- Bit2 gesetzt (Right):
 - Setze x-Wert= xmax



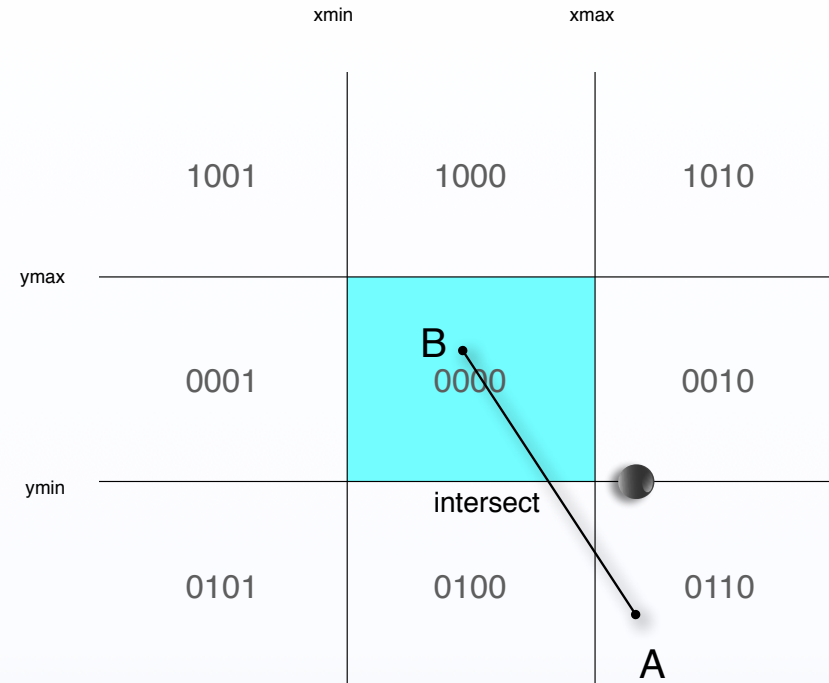
Cohen Sutherland Line Clipping

- Wie werden die Koordinaten des Schnittpunktes berechnet?
- Den Punkt auswählen dessen RegionCode ungleich 0000 ist (aussen): PUNKT A
- Bit1 gesetzt (Left):
 - Setze x-Wert= xmin
- Bit2 gesetzt (Right):
 - Setze x-Wert= xmax
- Bit3 gesetzt (Bottom):



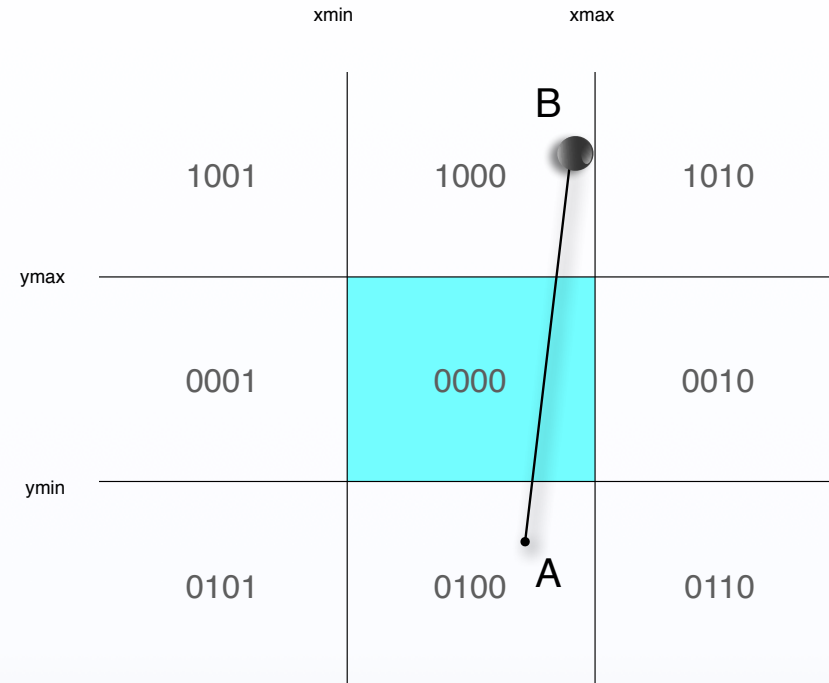
Cohen Sutherland Line Clipping

- Wie werden die Koordinaten des Schnittpunktes berechnet?
- Den Punkt auswählen dessen RegionCode ungleich 0000 ist (aussen): PUNKT A
- Bit1 gesetzt (Left):
 - Setze x-Wert= xmin
- Bit2 gesetzt (Right):
 - Setze x-Wert= xmax
- Bit3 gesetzt (Bottom):
 - Setze y-Wert= ymin



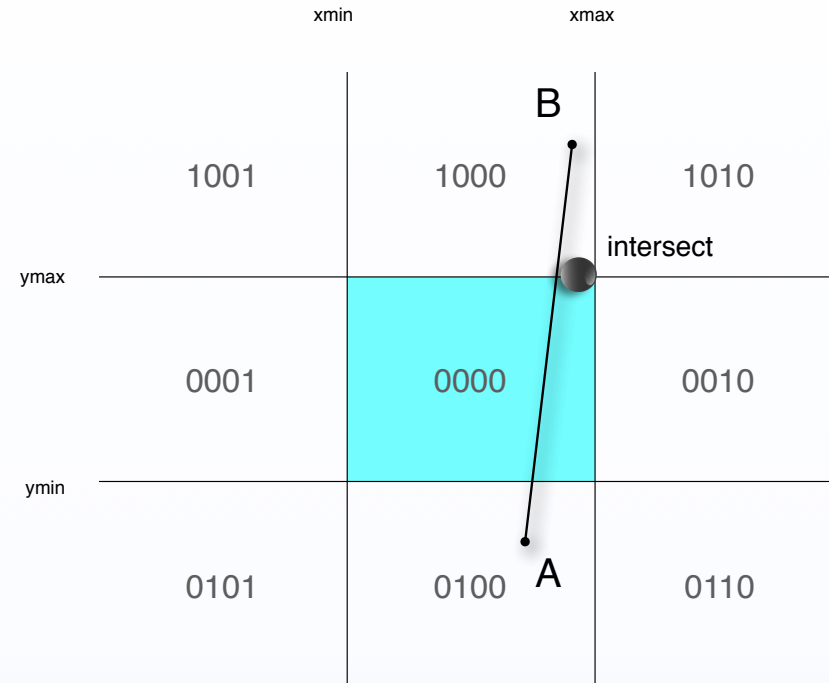
Cohen Sutherland Line Clipping

- Wie werden die Koordinaten des Schnittpunktes berechnet?
- Den Punkt auswählen dessen RegionCode ungleich 0000 ist (aussen): PUNKT A
- Bit1 gesetzt (Left):
 - Setze x-Wert= xmin
- Bit2 gesetzt (Right):
 - Setze x-Wert= xmax
- Bit3 gesetzt (Bottom):
 - Setze y-Wert= ymin
- Bit 4 gesetzt (Top):



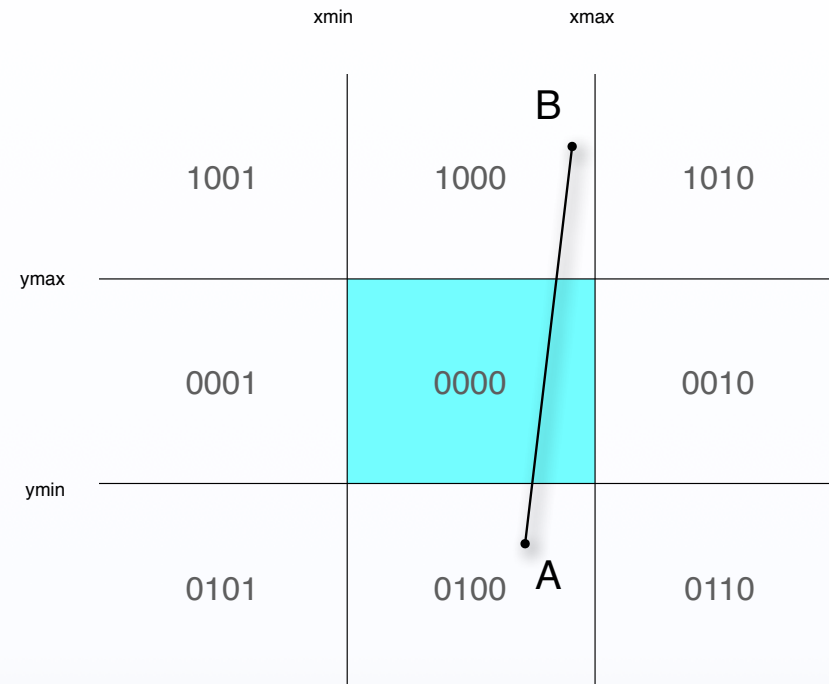
Cohen Sutherland Line Clipping

- Wie werden die Koordinaten des Schnittpunktes berechnet?
- Den Punkt auswählen dessen RegionCode ungleich 0000 ist (aussen): PUNKT A
- Bit1 gesetzt (Left):
 - Setze x-Wert= xmin
- Bit2 gesetzt (Right):
 - Setze x-Wert= xmax
- Bit3 gesetzt (Bottom):
 - Setze y-Wert= ymin
- Bit 4 gesetzt (Top):
 - Setze y-Wert= ymax
- Ist der Schnittpunkt damit berechnet?



Cohen Sutherland Line Clipping

- Wie werden die Koordinaten des Schnittpunktes berechnet?
- Den Punkt auswählen dessen RegionCode ungleich 0000 ist (aussen): PUNKT A
- Bit1 gesetzt (Left):
 - Setze x-Wert= xmin
- Bit2 gesetzt (Right):
 - Setze x-Wert= xmax
- Bit3 gesetzt (Bottom):
 - Setze y-Wert= ymin
- Bit 4 gesetzt (Top):
 - Setze y-Wert= ymax
- Ist der Schnittpunkt damit berechnet?
- Nur eine Koordinate des Punktes ist berechnet; die zweite muss noch berechnet werden!



Intersection Berechnung

Geradengleichung: $ax + by + c = 0$

Left: $x = x_{min}$

Right: $x = x_{max}$

$$y = -\frac{a \cdot x + c}{b}$$

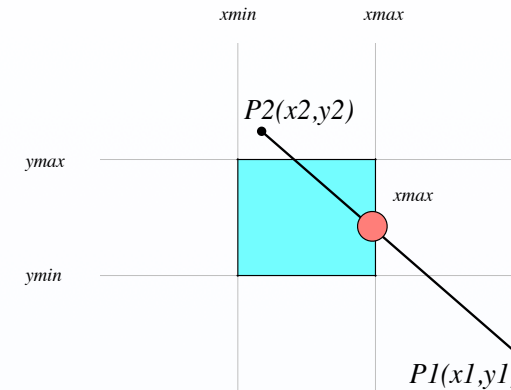
Bottom: $y = y_{min}$

Top: $y = y_{max}$

$$x = -\frac{b \cdot y + c}{a}$$

wie berechnen wir a, b, c ?

in Abhängigkeit von den geg. $(x1, y1)$ $(x2, y2)$



Intersection Berechnung

$$a \cdot x + b \cdot y + c = 0$$

Gerade parametrisch darstellen:

$$x = x_1 + t(x_2 - x_1)$$

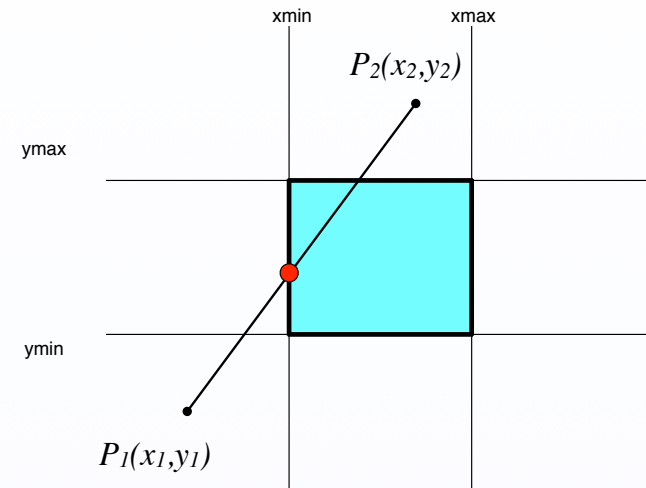
$$y = y_1 + t(y_2 - y_1)$$

$$t \text{ eliminieren: } t = \frac{x - x_1}{x_2 - x_1}$$

$$y = y_1 + \frac{x - x_1}{x_2 - x_1} \cdot (y_2 - y_1)$$

$$y(x_2 - x_1) = y_1(x_2 - x_1) + (x - x_1) \cdot (y_2 - y_1) \quad \text{ausmultiplizieren, umstellen}$$

$$x \cdot \underbrace{(y_1 - y_2)}_a + y \cdot \underbrace{(x_2 - x_1)}_b + \underbrace{x_1 \cdot y_2 - y_1 \cdot x_2}_c = 0$$



Cohen Sutherland Line Clipping

- Effiziente Implementierung für Clipping von Linien mit „Fenstern“
- Zwei-Stufige Arbeitsweise
 1. Für Start-, bzw. Endpunkt werden sog. RegionCodes ermittelt
 2. Durch einfache Bitoperationen, Linien ganz ausserhalb und Linien komplett innerhalb, schnell ermitteln
 3. Die Clipping-Kandidaten werden solange mit den Begrenzungen geschnitten (intersection) bis das resultierende Liniensegment innerhalb des Fensters liegt.
- Nachteile
 - RegionCode Berechnung gilt nur für rechteckige Clipping Fenster
 - Intersection Methode (Einsetzen der min, max Werte) gilt nur für rechteckige Fenster

```

void Cohen_Sutherland(int ax, int bx, int ay, int by){

    unsigned char c1= regionCode(ax,ay);
    unsigned char c2= regionCode(bx,by);
    unsigned char actual_c;
    float a,b,c,d,x,y;

    while ((c1 | c2) !=0){          // OR: both regionCodes inside

        if (( c1 & c2 ) !=0) return; // Both regionCodes outside

        // Line is clipping candidat
        // calculate constant terms for intersection calculation

        a = ay-by;
        b = bx-ax;
        d = ax*by-ay*bx;

        // select point lying outside

        if (c1==0)
            actual_c =c2;    // if c1 inside then select c2
        else
            actual_c =c1;    // c1 ist outside, or both are outside
    }
}

```

```

a = ay-by;
b = bx-ax;
d = ax*by-ay*bx;

// select point lying outside

if (c1==0)
    actual_c =c2;    // if c1 inside then select c2
else
    actual_c =c1;    // c1 ist outside, or both are outside

// sequential clipping

if ((actual_c & C_LEFT) !=0)                // C_LEFT=0001
    { x=XMIN; y= -(a*x+d)/b; }              // intersection
else if ((actual_c & C_RIGHT) !=0)
    { x=XMAX; y= -(a*x+d)/b; }
else if ((actual_c & C_TOP) !=0)
    { y=YMAX; x= -(b*y+d)/a; }
else if ((actual_c & C_BOTTOM) !=0)
    { y=YMIN; x= -(b*y+d)/a; }

// update regionCode
if (actual_c == c1)
    { ax=x; ay=y; c1=regionCode(ax,ay); } //update
else
    { bx=x; by=y; c2=regionCode(bx,by); } //update
} // endwhile

bresenham_line(ax,ay,bx,by);
} // end Cohensutherland

```